

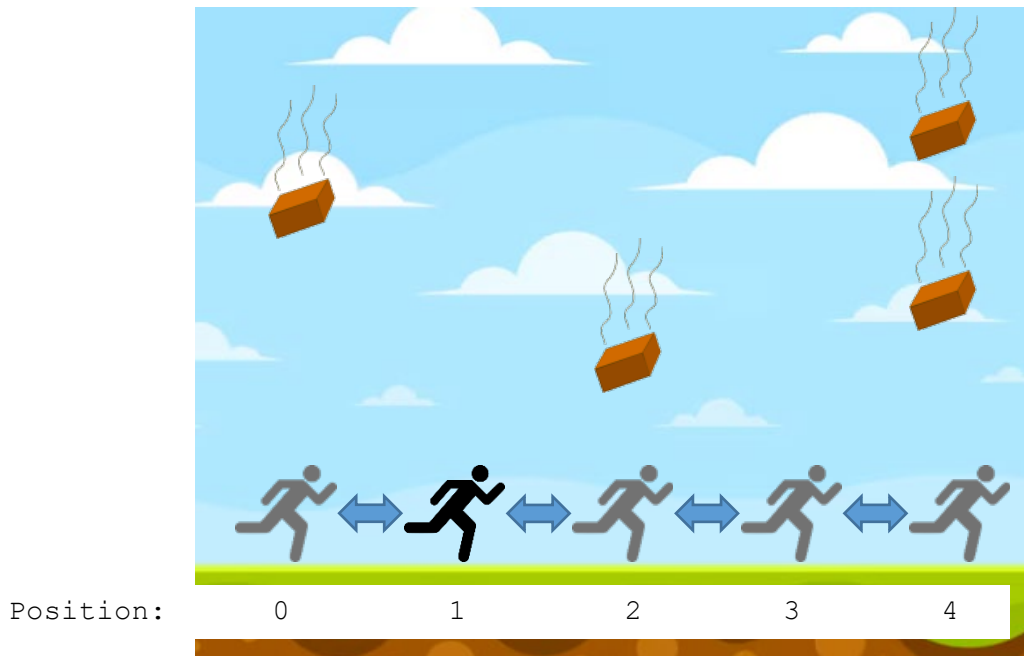
IT5001 Practical Exam AY2025/26 Sem 1

Instructions:

1. Your code will be graded and run on **Python 3.12**.
2. This exam consists of 3 parts, and each part has several tasks. You will submit **one code per part (one code may contain many functions)**, containing solutions to all tasks in that part.
3. Answer all questions printed in blue and black only.
4. Your code must adhere to the following.
 - Use the **same function name** and the **same function signature** as specified in the question. Do not add any new parameters.
 - You may write extra functions to structure your code better, unless the specific part or task explicitly forbids it. Do remember to copy any extra functions over to the specific part(s) they are supporting.
 - Do not import any packages or functions, unless the specific part or task explicitly allows it. However, you may use any Python built-in functions that do not require importing packages.
 - You should always `return` the output instead of `print()`.
 - Do not use decorators.
 - Do not mutate the input arguments.
5. **No marks will be given if your code cannot run**, including if it causes syntax errors or program crashes. We will just grade what you have submitted, and we will not fix your code.
6. Your code must be good and efficient and pass our hidden test cases for you to obtain full marks. Working code that produces correct answers in public test cases may or may not give you marks.
7. Marks will be deducted if your code is ***unnecessarily long, hard-coded, in a poor programming style, or includes irrelevant code***.
 - Do not include debugging or test code in your submission. Test cases should either be deleted or commented out in your submitted code.
 - Remove (or comment out) any lines of code that you do not want us to grade.
8. Submit your solution for each part as **separate, independent code**.
 - If you reuse function(s) from another part, you will need to copy them (along with their supporting functions, if any) into the part you are working on.
 - Any redundant functions for the part (albeit from a different part) will be deemed as "irrelevant code".
9. **You are not allowed to use any generative AI tool such as ChatGPT or Copilot in the assessment.**
10. Reminder: Any kind of **plagiarism** such as copying code with modifications will be caught. This includes adding comments, changing variable names, changing the order of lines/functions, adding useless statements, etc. Basically, you should write all your code from scratch.
11. Before the PE ends, remember to:
 - Save your 3 parts of code in 3 files on the computer.
 - Submit your code to Exemplify and make sure that they are the same as your saved copies.
 - **Do not make any more changes in your saved copies after Exemplify submission.**
 - Do not copy your code *FROM* Exemplify to submit to Coursemology. Instead, copy from IDLE directly.

Part 1: Dodge! (10+10+10+10 = 40 marks)

Let's develop a game! In this game, the player has to dodge falling bricks.



As illustrated above, the player can be at one of five discrete positions (0 to 4) at any one time. The player moves by pressing 'L' to go left or 'R' to go right. For example:

- If the player is at position 3, pressing 'R' (right) will change its position to 4.
- If the player is at position 1, pressing 'L' (left) will change its position to 0.

However, the player will NOT go out of the boundary. For example:

- If the player is already at position 4, pressing 'R' (right) will not change the position and it will remain at position 4.

Task 1: Next Position (10 marks)

Write a function `next_pos(curr_pos, move)`, where:

- `curr_pos` is the current player position between 0 and 4 inclusive
- `move` is either 'L' or 'R'

to return the new position in **integer** after the move.

Rules for this task: Write one function only, and do not call other functions.

Examples:

```
>>> print(next_pos(3, 'L'))
2
>>> print(next_pos(3, 'R'))
4
>>> print(next_pos(4, 'R'))
4
```

Task 2: Final Position - Iterative version (10 marks)

Write an iterative function `final_pos_i(curr_pos, moves)`, where:

- `curr_pos` is the current player position between 0 and 4 inclusive
- `moves` is a string of characters that are either 'L' or 'R' and no other character

to return the final position after the sequence of player moves represented by `moves`.

Rules for this task: Your function must use for-loops or while-loops and cannot be recursive. You may reuse the function you wrote for Task 1.

Examples:

```
>>> print(final_pos_i(2, 'LLR'))
1
>>> print(final_pos_i(4, 'LRRRL'))
3
```

Task 3: Final Position - Recursive version (10 marks)

Write a recursive function `final_pos_r(curr_pos, moves)` with the same functionality as `final_pos_i(curr_pos, moves)` in the previous task.

Rules for this task: Your function must be purely recursive. You cannot use any loop or list comprehension. You may reuse the function you wrote for Task 1.

Task 4: Play Game (10 marks)

The game has falling bricks that the player must dodge to earn scores. For simplicity, we only consider the point of time when a brick reaches the ground (and either hits or misses the player), and we define “time” in reference to the sequence of player moves.

Each brick will fall on one of the five discrete positions (0 to 4) at a certain time. We use `bricks`, a string of digits (0 to 4) to represent the positions that are hit by the falling bricks at each point in the sequence, corresponding to the sequence of player moves.

For example, if we have `curr_pos = 4`, `moves = 'LRRRL'`, and `bricks = '14342'`, this means:

- Before “time” starts, the player is at position 4 (i.e., `curr_pos`).
- At time 0, the player has moved left ('L', i.e., first move in the sequence) to position 3.
- At the same time 0, a brick falls at position 1 (i.e., first digit in the sequence). It misses the player (who is at position 3).
- At time 1, the player has moved right ('R') to position 4.
- At the same time 1, another brick falls at position 4. It hits the player (who is also at position 4), ending the game.

The player earns a score of **100** for every brick dodged. If the player completes the full sequence of moves without getting hit by any brick, **the total score is doubled** as a bonus.

The game ends when either:

- A brick hits the player (i.e., at the earliest time this happens), or
- The sequence of player moves is completed without any hit.

For the above example, the game ends with a score of 100 for dodging only the first brick.

Write a function `play_game(curr_pos, moves, bricks)` that returns the score at the end of the game. You can assume that `moves` and `bricks` are of the same length, and that `bricks` contains no other character than 0 to 4.

You may reuse the functions you wrote for Tasks 1-3 in this Part 1.

Examples:

```
>>> print(play_game(2, 'LLR', '024')) # dodged all 3 bricks and doubled
600
>>> print(play_game(4, 'LRRRL', '14342')) # dodged only 1 brick
100
>>> print(play_game(4, 'LRRRL', '12341')) # dodged only 3 bricks
300
>>> print(play_game(4, 'LRRRL', '01234')) # dodged all 5 bricks and doubled
1000
```

Part 2: Looting the Maze! (20+15 = 35 marks)

You are given a maze layout in the matrix form, like the maze in our tutorials. For example:

```
maze0 = [['0', '0', '1', '1', '0', 'P'],
          ['0', 'P', '1', '1', '0', '0'],
          ['0', '1', '1', '1', '1', '0'],
          ['1', 'P', 'P', '1', '1', '0'],
          ['0', '0', '0', '1', '1', 'P']]
```

In this maze representation:

- Cells with '1' are walls that you cannot penetrate.
- Cells with '0' and 'P' are spaces that you can go through.
- Further, a cell with 'P' has a prize that you can loot.

The maze exploration rules are similar to the ones in the tutorial:

- You can move to '0' or 'P' cells, but you cannot move to '1' cells.
- From a certain position, you can only move north, east, south, or west to the direct 4 neighboring cells. You cannot move diagonally to e.g., the north-east cell. And of course, you cannot move out of the maze boundary.

You can explore the maze from different starting positions, which will allow you to loot different numbers of prizes.

These examples show you the prizes you can get (indicated with green rectangles) in the above maze:

```
00110P
0P1100
011110
1PP110
00011P
```

If you start at position (0,0),
you can only get 1 prize.

```
00110P
0P1100
011110
1PP110
00011P
```

If you start at position (4,0),
you can get 2 prizes.

```
00110P
0P1100
011110
1PP110
00011P
```

If you start at position (1,4),
you can get 2 prizes.

If the starting position is outside the maze or is on a '1' cell, you will get 0 prize.

Task 1: Treasure Hunt (20 marks)

Write a function `hunt_treasure(maze, start_pos)` where:

- `maze` is a 2D array of '0', '1' and 'P' values representing the maze layout
- `start_pos` is a tuple of matrix indices representing the starting position

and return the number of prizes you can loot with the specified starting position.

Examples:

```
>>> maze0 = [['0', '0', '1', '1', '0', 'P'],
              ['0', 'P', '1', '1', '0', '0'],
              ['0', '1', '1', '1', '1', '0'],
              ['1', 'P', 'P', '1', '1', '0'],
              ['0', '0', '0', '1', '1', 'P']]
>>> print(hunt_treasure(maze0, (0,0)))
1
>>> print(hunt_treasure(maze0, (4,0)))
2
>>> print(hunt_treasure(maze0, (1,4)))
2
```

Task 2: Penetrating One Wall (15 marks)

Wow, you now get some magic that lets you penetrate a wall temporarily! However, you can penetrate only one wall at a time, namely, you cannot consecutively penetrate two or more walls. However, after penetrating one wall, you can rest on a non-wall cell and penetrate another wall later. In other words, you can only penetrate a wall that can lead you to a non-wall cell in any direction.

For example, for the above maze:

- From (2,0), you can penetrate the south wall (3,0) to go to (4,0) or (3,1) in the next step.
- However, from (0,1), you cannot penetrate the east wall (0,2), since you cannot consecutively penetrate the wall at (0,3) or (1,2).

```
00110P
0P1100
011110
1PP110
00011P
```

Allowed

```
00110P
0P1100
011110
1PP110
00011P
```

Allowed

```
00110P
0P1100
011110
1PP110
00011P
```

Disallowed

Using your wall penetration magic, you may be able to get more prizes!

```

00110P
0P1100
011110
1PP110
00011P

```

If you start at position (0,0),
you can get **3** prizes.

```

00110P
0P1100
011110
1PP110
00011P

```

If you start at position (4,0),
you can get **3** prizes.

```

00110P
0P1100
011110
1PP110
00011P

```

If you start at position (1,4),
you can get **2** prizes.

```

011P000
0011111
0011111
000P110
1111000
1111000
1111111
P000000

```

If you start at position (7,0), which
has a prize, you can get **2** prizes.

Write a function `hunt_treasure_penetrate_one_wall(maze, start_pos)`, where `maze` and `start_pos` are as defined in the previous task, and return the number of prizes you can loot with the wall penetration magic.

Example:

```

>>> maze0b = [['0', '1', '1', 'P', '0', '0', '0'],
               ['0', '0', '1', '1', '1', '1', '1'],
               ['0', '0', '1', '1', '1', '1', '1'],
               ['0', '0', '0', 'P', '1', '1', '0'],
               ['1', '1', '1', '1', '0', '0', '0'],
               ['1', '1', '1', '1', '0', '0', '0'],
               ['1', '1', '1', '1', '1', '1', '1'],
               ['P', '0', '0', '0', '0', '0', '0']]
>>> print(hunt_treasure_penetrate_one_wall(maze0b, (7,0)))
2

```

Part 3: Halloween Candies (25 marks)

For this part, you are allowed to import any default Python packages (that do not require extra download).

It's Halloween night! After hours of trick-or-treating, you return home with a big basket filled with candies of all shapes and sizes. Your twin children are waiting eagerly — but to keep things fair, you want to divide all the candies equally (by weight) between them. You cannot break or split any candy — each candy must go entirely to one child or the other.

You want to know if it is possible to divide all the candies into two groups, so that each of your twins gets exactly the same total weight of candy.

Write a function `divisible(candies)` that accepts a list of integers `candies` containing the weights of the candies in grams. Return `'YES'` if it is possible to divide all the candies so that both twins receive the same total weight; otherwise, return `'NO'`. (Do not “print” your answer.)

You can assume the input list is non-empty and only contains integers.

Examples:

```
>>> print(divisible([1,1,2,4])) # divide the list into [1,1,2] and [4]
YES
>>> print(divisible([10,1,1,2,4,7,8])) #impossible to be divided equally
NO
```

Note that you will only get full marks for this part if your code can handle large inputs within 1 second.

```
>>> print(divisible([i for i in range(1,200)]))
YES
```

Code Template

Part 1

```
def next_pos(curr_pos,move):  
    return -1
```

```
def final_pos_i(curr_pos,moves):  
    return -1
```

```
def final_pos_r(curr_pos,moves):  
    return -1
```

```
def play_game(curr_pos,moves,bricks):  
    return -1
```

Test cases below; do not include in your submission.

```
print(next_pos(3,'L')) # Expected: 2  
print(next_pos(3,'R')) # Expected: 4  
print(next_pos(4,'R')) # Expected: 4
```

```
print(final_pos_i(2,'LLR')) # Expected: 1  
print(final_pos_i(4,'LRRRL')) # Expected: 3
```

```
print(final_pos_r(2,'LLR')) # Expected: 1  
print(final_pos_r(4,'LRRRL')) # Expected: 3
```

```
print(play_game(2,'LLR','024')) # Expected: 600  
print(play_game(4,'LRRRL','14342')) # Expected: 100  
print(play_game(4,'LRRRL','12341')) # Expected: 300  
print(play_game(4,'LRRRL','01234')) # Expected: 1000
```

Part 2

```
def hunt_treasure(maze,start_pos):  
    return -1
```

```
def hunt_treasure_penetrate_one_wall(maze,start_pos):  
    return -1
```

Test cases below; do not include in your submission.

```
maze0 = [['0', '0', '1', '1', '0', 'P'],  
         ['0', 'P', '1', '1', '0', '0'],  
         ['0', '1', '1', '1', '1', '0'],  
         ['1', 'P', 'P', '1', '1', '0'],  
         ['0', '0', '0', '1', '1', 'P']]  
maze0b = [['0', '1', '1', 'P', '0', '0', '0'],  
          ['0', '0', '1', '1', '1', '1', '1'],  
          ['0', '0', '1', '1', '1', '1', '1'],  
          ['0', '0', '0', 'P', '1', '1', '0'],  
          ['1', '1', '1', '1', '0', '0', '0'],  
          ['1', '1', '1', '1', '0', '0', '0'],  
          ['1', '1', '1', '1', '1', '1', '1'],  
          ['P', '0', '0', '0', '0', '0', '0']]
```

```
print(hunt_treasure(maze0,(0,0))) # Expected: 1
print(hunt_treasure(maze0,(4,0))) # Expected: 2
print(hunt_treasure(maze0,(1,4))) # Expected: 2

print(hunt_treasure_penetrate_one_wall(maze0,(0,0))) # Expected: 3
print(hunt_treasure_penetrate_one_wall(maze0,(4,0))) # Expected: 3
print(hunt_treasure_penetrate_one_wall(maze0,(1,4))) # Expected: 2
print(hunt_treasure_penetrate_one_wall(maze0b,(7,0))) # Expected: 2
```

Part 3

```
def divisible(candies):
    return ''
```

Test cases below; do not include in your submission.

```
print(divisible([1,1,2,4])) # Expected: YES
print(divisible([10,1,1,2,4,7,8])) # Expected: NO

print(divisible([i for i in range(1,200)])) # Expected: YES
```